

Nothing Crashed — Instructor Notes

Class 14 (alternative) · review of Classes 9–13 and Lab 7 · 30 min · handout + skeleton notebook

The idea. An intern’s analysis contains three mistakes, and none of them produces an error message. Students predict on paper before they run each cell, then fix each mistake with the tools they already have: group, where, sample, a simulation loop and a p-value. The lesson is that a clean run tells you nothing about whether an analysis is right. The habits that catch the mistakes are predicting before running, asking “compared to what?”, looking at the sample and running loops at 10 first.

What students produce. A verdict on each of two claims, with corrected numbers, an explanation of why the intern’s test could only reject, and one claim rewritten honestly. **What this does not claim to teach:** debugging technique in general, new statistical methods, or anything about real transit data. Riverton and its data are invented, and the handout says so.

New material: none. One gotcha is made explicit. `Table.sample` draws with replacement by default, which is the same trap as `np.random.choice` in Class 13.

Materials. Handout, skeleton notebook, `data/riverton_trips.csv`. The notebook reads from `data/`, so upload it that way on the hub.

Timing (30 min)

Segment	Min	What happens
Part 1	3	Read the report, Q1–2 individually, compare. Stop bar.
Exhibit A	6	Q3 prediction, run, totals, Blue by hand, Q4.
Exhibit B	6	Q5 prediction, dates, random sample, Q6.
Exhibit C	10	C.1 one sample, Q7 trace, intern’s run, run at 10, fix, Q8.
Part 3	5	Q9–10 in teams, then one minute out loud on which habit caught each exhibit.

Watch out. Never cut C: Q9 is the payoff. If a team hasn’t started B by minute 12, have them run B.3 with `False` filled in so they get to C.

Note. Cut for time (October 2026). An earlier draft had a third claim (“the average bus arrives 3.6 minutes late”). It came from averaging only trips with `minutes_late > 0`; the true mean is 1.54, and the filter dropped 209 of 620 trips. It is in the generator data if you want it back as a warm-up.

The numbers

All computed in the local `ds313` environment from `riverton_trips.csv`: 620 trips, sorted by date, every trip Riverton “ran” in 2025. Here, “on time” means `minutes_late <= 5`.

Trips per line	Blue 300, Red 150, Green 100, Gold 70
Late trips (> 5 min)	Blue 41 , Red 26, Green 25, Gold 22
Fraction late	Blue 0.137 , Red 0.173, Green 0.250, Gold 0.314
On time, full year	506 of 620 = 0.816 . The 80% claim is true for 2025.

Intern's first 80 rows	2025-01-01 to 2025-02-14 (58 January, 22 February); 51 on time (0.64)
On-time rate by month	Jan 0.60, Feb 0.78, Mar 0.92, Jul 0.92, Dec 0.58. Winter is the slow season.

	Random sample (False)	Random sample (True)
on time out of 80, 5th–95th pct	60–70 (median 65)	59–71 (median 65)
smallest / largest seen	53 / 78	51 / 78
median p-value (loop fixed)	0.65	0.65
fraction of samples with $p < 0.05$	0.011	0.016

20,000 random samples each, with p-values computed exactly from the binomial; the notebook's 10,000-repetition simulation agrees to about ± 0.01 . With replacement, a sample of 80 repeats about 5 trips on average, and at least one trip 99.5% of the time. The null distribution (80 trips at 80%) has mean 64, SD 3.6, and its middle 95% runs 57–71. With the intern's sample (51) and the loop fixed, the exact p-value is 0.0005. So fixing C alone does not rescue Claim 2; B must be fixed too.

Note. The buggy loop in C can only output 0 or 0.0001. simulated holds one number. That number is either $\leq$ observed (giving $1 / 10000$) or it isn't (giving 0). With a random sample, about 64% of teams get 0.0001 and the rest get 0.0. Either way the intern “rejects” at any cutoff. With the intern's own first-80 sample it is 0.0 essentially always.

Part 1. The Report

(≈ 3 min)

- 1 Anything, committed. Expect Claim 2 to draw more suspicion (“ $p = 0.0$ ” looks too clean). Claim 1 sounds like plain counting, and it is the one whose ranking flips completely.
- 2 Any specific mechanism counts (“only counted some buses,” “the sample wasn't random”). Push vague answers (“the data could be wrong”) toward a specific mechanism.

Exhibit A. Counts are not rates

(≈ 6 min)

- 3 Blue runs the most trips (300 of 620, almost half), so it can pile up the most late trips while being late the **least often**. Any “it runs more buses” answer is right.
- 4 By hand, $41 \div 300 = 0.137$. From the notebook: Blue 0.137, Red 0.173, Green 0.250, Gold 0.314. **FALLS**. The worst line is Gold. Blue is the **best** line. The ranking flips, rather than just shuffling.

Note. A.3 works because group sorts both tables alphabetically, so count columns line up row for row. That is worth one sentence out loud. If a team had a line with zero late trips, it would vanish from `late_trips.group('line')`, and the division would misalign or fail. Here every line has late trips, so it doesn't come up.

Exhibit B. Sorted data is not a sample

(≈ 6 min)

5 The first few weeks of January, because the table is sorted by date and 80 is about one-eighth of 620. That is the snowiest stretch of the year, when buses run late. Accept any version of “winter” or “only one season.”

6 2025-01-01 to 2025-02-14. **FALSE**: each trip happened once, so the sample should hold each trip at most once. A typical random sample has 60–70 of 80 on time.

Watch out. If a team runs B.3 with the . . . still in place, Python treats . . . as true, so sample silently draws **with** replacement and there is no error. That is on theme. The numbers barely change (table above), so if a team circles TRUE, the point to make is about the trips, not the p-value. Each trip ran once, so a sample that contains it twice describes a year that didn’t happen.

Exhibit C. The loop that ran once

(≈ 10 min)

7 The simulated column reads empty, empty, empty, then [61]. The np.append line is not indented, so it is outside the loop and runs once, after the loop ends, with the last one. With 10,000 repetitions simulated still holds 1 number.

8 Varies by team: for 90% of samples it falls between 0.16 and 0.97 (median about 0.65). **STANDS**, in the sense that there is no evidence against it. Not “proven.”

Watch out. If a team skipped B.3, observed is still the intern’s 51, and the fixed loop gives about 0.0005. They will think the claim falls. Ask them which sample their p-value describes.

Note. In C.3 the intern’s loop at 10 prints a one-element array. That is the moment the bug becomes visible, and it is our “10 before 10,000” rule paying off in someone else’s code. The histogram in the last cell is a second check: a correct null distribution is a hump centred near 64, not a single bar.

Part 3. The Verdict

(≈ 5 min)

Note. Close out loud (1 min). Run down the exhibits and ask which habit would have caught each mistake before the report went out. A: ask “compared to what?” and turn counts into rates. B: look at the sample (first and last rows) and draw at random. C: run loops at 10 first and check len(simulated). The common thread is that every mistake was caught by a **prediction** failing, not by an error message. A was in the sentence; B and C were in the code.

9 0.0001. The count in the numerator is 0 or 1, so the only possible p-values are 0 and 0.0001, and both are tiny. The test could never have failed to reject at any cutoff. Its conclusion was fixed before any data went in. If a test can only come out one way, it isn’t a test.

10 Example: “In a random sample of 80 trips, 65 arrived within 5 minutes. If the 80% claim were true, a count this low would happen about 65% of the time ($p \approx 0.65$), so the sample gives no evidence against the claim.” Stronger: “506 of the 620 trips in 2025 (82%) arrived within 5 minutes.”

If You Finish Early

A Nothing new. About 1% of random samples give $p < 0.05$ here. Even if the on-time rate were exactly 80%, about 4% would (counts of 57 or fewer). With ten teams, one rejection is roughly what chance predicts. Reporting only that team's result would be the intern's mistake again, through selection this time.

B No. The table **is** the population for 2025. Count it: $506 / 620 = 81.6\%$, and Claim 2 holds for 2025. A test only makes sense if the question is about the **process**, for example whether 2025 is consistent with a system that is on time 80% of the time in general, including future years. Under that model $P(X \leq 506) \approx 0.85$, so 2025 looks entirely ordinary for an 80% system.

Appendix: solution code (instructor only)

```
# A.3
fraction_late = late_counts.column('count') / all_trips.column('count')

# B.3
sample = trips.sample(80, with_replacement=False)

# C.4 (then change 10 to 10000)
simulated = make_array()
for i in np.arange(10000):
    one = claim.sample_from_distribution('Chance', 80).column('Chance
sample').item(0)
    simulated = np.append(simulated, one)
```

The dataset was generated by a seeded script (seed 1414). It applies line, winter-month and an unrecorded rain effect, plus normal noise (SD 3.5 min). Rain is deliberately not a column.